@thesanskaarsingh

Code Today,
Build Tomorrow,
Inspire Forever.

- Learn.
- Build.
- Deploy.
- Repeat.

REACT.JS NOTES

From Basics to Beautiful UI!

WHAT YOU'LL LEARN

01	02	03	04	05	06	07	08
React Essentials	Components & JSX	Props & State	Hooks (useState, useEffect)	Event Handling & Forms	Conditional Rendering	React Router (Dynamic Apps)	Context API & State Management

KEY TAKEAWAYS!

- Component Based UI
- Reusability is Power
- Think Declaratively
- One-way Data Flow
- Hooks make it Simple
- Build Once, Use Anywhere!

```jsx
import React from 'react';

function App() {
  const [count, setCount] =
   React.useState(0);

  return (
    <div className="app">
    <h1>Hello React! ♥ </h1>
    <p>Count: {count}</p>
    <button onClick={() =>
     setCount(count + 1)}>
      Increment
    </button>
  </div>
);
export default App;
```

PERFECT FOR:

- Students
- Developers
- Job Aspirants
- Problem Solvers
- Creative Thinkers
- Building Amazing UIs!

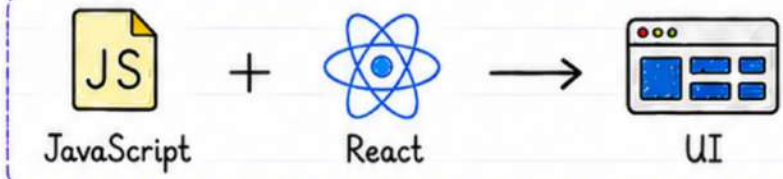
Understand Deeply • Practice Consistently • Build Confidently • Deploy Boldly • Keep Improving Everyday

Let's Build Something Amazing with React!

01. INTRODUCTION TO REACT

What is React?

React is a JavaScript library for building user interfaces, especially Single Page Applications (SPA).



Why React?

- Component Based Architecture
- Reusable UI Components
- Virtual DOM for Better Performance
- Strong Community Support
- Learn Once, Use Anywhere

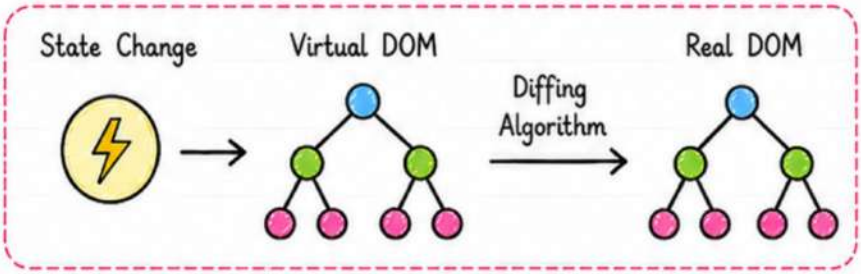
Used by:

Key Features

- Declarative UI
- Component-Based
- Virtual DOM
- One-way Data Binding
- JSX Syntax
- Reusable Components
- Hooks (useState, useEffect, etc.)
- Fast Rendering
- Easy to Test & Maintain

Virtual DOM

React creates a lightweight copy of the real DOM and updates only the changed parts. This makes React very fast!



SPA (Single Page Application)

React helps in building SPA where only one HTML page is loaded and content is updated dynamically.

Component-Based Architecture

UI is divided into small, independent components. Each component manages its own logic and can be reused anywhere.

Key Takeaway: React makes building modern, fast and scalable UI easy with components, JSX and powerful features.

02. JSX & COMPONENTS

What is JSX?

JSX (JavaScript XML) is a syntax extension for JavaScript.
It looks like HTML, but it is transpiled to React.createElement() calls by Babel.

HTML

```
<h1 class="title">
  Hello World
</h1>
```

JSX

```
<h1 className="title">
  Hello World
</h1>
```

- Note: In JSX, use className instead of class

Rules of JSX

- Return a single parent element.
- Use className instead of class.
- Use camelCase for HTML attributes.
- Always close tags.
- JSX expressions are written inside {}.

Component Naming

Always start component names with a capital letter.

✅ MyComponent ❌ mycomponent

Functional Components

In React, components are the building blocks of UI. Functional components are simple JavaScript functions that return JSX.

```
function Hello() {
  return (
    <h1>Hello React!</h1>
  );
}
export default Hello;
```

Returning JSX

A component must return JSX.

```
function Welcome() {
  return <h2>Welcome to React!</h2>;
}
```

Props Introduction

Props (short for "properties") are used to pass data from one component to another.

```
function Greet(props) {
  return <p>Hello, {props.name}!</p>;
}
// Usage
<Greet name="Sanskaar" />
```

JSX vs HTML

HTML	JSX
class	className
for	htmlFor
onclick	onClick
style="color:red"	style={{ color: 'red' }}

Key Takeaway

JSX makes React code more readable and easy to write. Components help us build reusable and maintainable UI. ❤️

03. PROPS & STATE

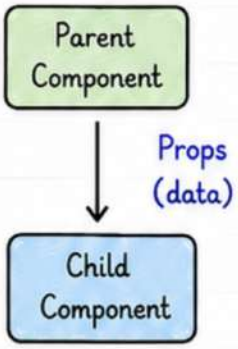
What are Props?

- Props (short for "properties") are used to pass data from one component to another.
- Props are read-only (immutable).
- Data flows in one direction (parent → child).

Example:

```
function User(props) {
  return <p>Hello, {props.name}!</p>;
}

// Parent Component
<User name="Sanskaar" />
```

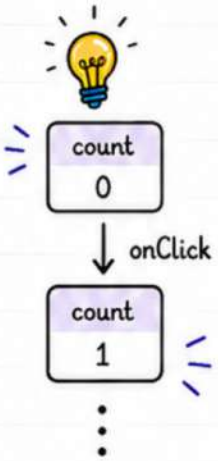


What is State?

- State is data managed within a component.
- It can change over time.
- When state changes, component re-renders.
- State is private to the component.

Example:

```
import { useState } from 'react';
function Counter() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>
        Increase
      </button>
    </div>
);
}
```

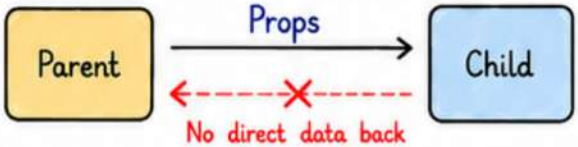


Props vs State

Feature	Props	State
Owner	Parent	Component
Mutability	Immutable (read-only)	Mutable (can change)
Purpose	Pass data	Manage data
Updated by	Parent	Component itself
Re-render	When props change	When state changes

One-way Data Flow

Data flows from parent to child using props.
Child cannot send data back directly.

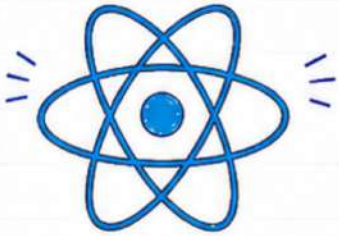


Best Practices

- Use props for passing data.
- Keep state as minimal as possible.
- Lift state up when multiple components need the same data.
- Avoid mutating state directly.

Key Takeaway

Props pass data.
State manages data.
Together they build dynamic and interactive React applications!



04. HOOKS

Hooks are special functions that let you use state and other React features in functional components.

1. useState()

- Adds state to functional components.
- Returns a state variable and a function to update it.
- Triggers re-render on update.

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>
        Increase
      </button>
    </div>
  );
}
```

2. useEffect()

- Runs side effects in components.
- Runs after every render by default.
- Used for API calls, subscriptions, DOM updates, etc.

```
import { useState, useEffect } from 'react';

function App() {
  const [data, setData] = useState([]);

  useEffect(() => {
    fetch('https://api.example.com/data')
      .then(res => res.json())
      .then(setData);
  }, []);   // Empty array = run once

  return <div>{data.length} items</div>;
}
```

3. useRef()

- Provides a mutable ref object.
- Does not cause re-render when updated.
- Commonly used for DOM access.

```
import { useRef } from 'react';

function FocusInput() {
  const inputRef = useRef(null);

  return (
    <div>
      <input ref={inputRef} />
      <button onClick={() =>
        inputRef.current.focus()}>
        Focus Input
      </button>
    </div>
  );
}
```

4. useMemo()

- Memoizes a value.
- Recomputes only when dependencies change.
- Optimizes expensive calculations.

```
const memoValue = useMemo(() => {
  return expensiveCalculation(a, b);
}, [a, b]);
```

5. useCallback()

- Memoizes a function.
- Returns the same function reference until dependencies change.
- Useful for child components to prevent re-renders.

```
const handleClick = useCallback(() => {
  console.log('Clicked!');
}, [count]);
```

6. Hook Rules

- Only call Hooks at the top level.
- Do not call Hooks inside loops, conditions or nested functions.
- Only call Hooks from React functional components or custom Hooks.

Example: Using Multiple Hooks

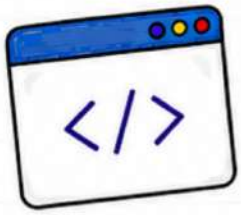
```
function Dashboard() {
  const [count, setCount] = useState(0);
  const inputRef = useRef(null);
  useEffect(() => { console.log('Component mounted'), }, []);

  return <p ref={inputRef}>Count: {count}</p>;
}
```

Key Takeaway

- Hooks make functional components powerful and reusable.
- They help manage state, side effects and performance easily.
- Master Hooks to write clean and efficient React code!

Learn → Build → Debug → Repeat → Master React



05. EVENTS & CONDITIONAL RENDERING

1. EVENT HANDLING

- React uses camelCase for events.
- Events are passed as functions.
- We don't use quotes around function name.

onClick={handleClick} (without quotes)

```
function Button() {
  const handleClick = () => {
    alert('Button Clicked!');
  };

  return <button onClick={handleClick}>
    Click Me
  </button>;
}
```

Common Events

- → onClick – mouse click
- → onChange – input change
- → onSubmit – form submit
- → onMouseOver – mouse over
- → onFocus – input focus
- → onBlur – input blur
- → onKeyDown – key press down

2. onChange (For Inputs)

```
function NameInput() {
  const [name, setName] = useState("");

  return (
    <div>
      <input type="text" value={name}
        onChange={(e) => setName(e.target.value)}
        placeholder="Enter your name" />
      <p>Hello, {name}!</p>
    </div>
  );
}
```

e.target.value gives the current value of the input field.

Simple Form with onSubmit

```
function LoginForm(e) {
  const [email, setEmail] = useState("");
  const handleSubmit = (e) => {
    e.preventDefault();   // prevent page reload
    alert('Submitted: ' + email);
  };
  return (
    <form onSubmit={handleSubmit}>
      <input type="email" value={email}
        onChange={(e) => setEmail(e.target.value)}
        placeholder="Email" />
      <button type="submit">Submit</button>
    </form>
  );
}
```

3. CONDITIONAL RENDERING

- Used to show different UI based on condition.
- Ways to implement:

1. if Statement

```
if (isLoggedIn) {
  return <p>Welcome!
    </p>;
} else {
  return <p>Please
      Login</p>;
}
```

2. Logical AND (&&)

```
{isLoggedIn &&
  <p>Welcome Back!
    </p> }
```

Renders only if isLoggedIn is true.

3. Ternary Operator

```
return (
  <div>
   {isLoggedIn ?
    <p>Welcome!</p> :
    <p>Please Login</p>}
  </div>
);
```

4. LISTS & RENDERING WITH MAP()

- Used to render lists of data.
- Always add a unique key for each item.

```
const fruits = ['Apple', 'Banana', 'Mango'];
return (
  <ul>
    {fruits.map((fruit, index) => (
      <li key={index}> 🍎 {fruit} </li>
    ))}
  </ul>
);
```

Key

- Key helps React identify which items have changed, added or removed.
- Use unique id if available.

Key Takeaway

- Events make UI interactive.
- Conditional rendering helps show the right content.
- Lists + map() help display dynamic data efficiently.

06. REACT ROUTER

React Router is the standard routing library for React. It helps us navigate between pages (components) without reloading the entire application.

1. What is Routing?

- Routing is the process of showing different pages/components based on the URL.
- In React apps, routing is done on the client side using React Router.

2. Installation

```
npm install react-router-dom        (v6+)
```

3. Basic Setup

```jsx
import { BrowserRouter, Routes, Route } from 'react-router-dom';
import Home from './pages/Home';
import About from './pages/About';
import Contact from './pages/Contact';

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/contact" element={<Contact />} />
      </Routes>
    </BrowserRouter>
  );
}
```

4. Important Components

- BrowserRouter - Wraps the app & enables routing.
- Routes - Container for all route definitions.
- Route - Defines a route and the component to render.
- Link - Used to navigate between routes.
- useNavigate - Hook for programmatic navigation.
- useParams - Hook to get route parameters.

Example: Link

```jsx
import { Link } from 'react-router-dom';
<nav>
  <Link to="/">Home</Link> |
  <Link to="/about">About</Link> |
  <Link to="/contact">Contact</Link>
</nav>
```

Link is used instead of <a> tag to avoid page reload.

5. useNavigate Hook

```jsx
import { useNavigate } from 'react-router-dom';
function Home() {
  const navigate = useNavigate();
  return (
    <button onClick={() => navigate('/about')}>
      Go to About
    </button>
  );
}
```

Use navigate() to go to another route in code (like after form submit, login, logout, etc.)

6. Dynamic Routes (with Parameters)

```jsx
<Route path="/user/:id" element={<User />} />
// In User.jsx
import { useParams } from 'react-router-dom';
function User() {
  const { id } = useParams();
  return <p>User ID: {id}</p>;
}
```

:id is a dynamic parameter. We can access it using useParams().

7. Nested Routes (Simple)

```jsx
<Route path="/dashboard" element={<Dashboard />}>
  <Route path="profile" element={<Profile />} />
  <Route path="settings" element={<Settings />} />
</Route>
```

Routes can be nested inside other routes.

Key Takeaway

- React Router helps build SPA (Single Page Applications).
- Use Link for navigation and useNavigate for actions.
- Routes match the URL and show the right component.
- It makes navigation smooth without reloading the page.

07. STATE MANAGEMENT

State is data that changes over time and affects what is shown on the screen. Proper state management makes our app scalable and maintainable.

1. Local State (useState)

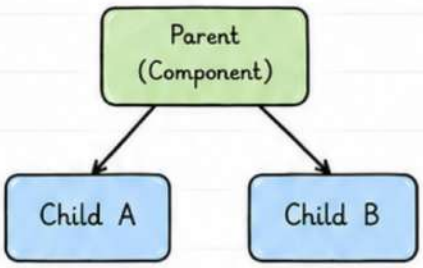
- State is managed within a single component.
- Simple and easy to use.
- Best for small, isolated state.

```
import { useState } from 'react';
function Counter() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <p>{count}</p>
      <button onClick={() => setCount(count +1)}>
        Increase
      </button>
    </div>
  );
}
```

State is local to Counter component only.

2. Lifting State Up

- When multiple components need the same state, lift it up to their common parent.
- Helps in sharing state between components.



Parent holds the state and passes it as props to children.

3. Context API

- Used to share global state across the component tree without prop drilling.
- Good for theme, user info, language, etc.

```
const UserContext = createContext();
function App() {
  return (
    <UserContext.Provider value={user}>
      <Navbar />
      <Profile />
    </UserContext.Provider>
  );
}
```

Use Context to provide and consume global data.

4. useContext()

- Hook to consume data from Context.
- Makes accessing global state easy.

```
const user = useContext(UserContext);
return <p>Welcome {user.name}</p>;
```

Use useContext() inside functional components.

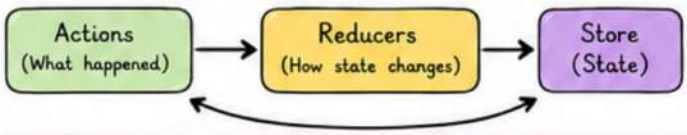
5. useReducer Hook

- An alternative to useState for complex state logic.
- Great for forms, carts, or multiple state transitions.

```
const [state, dispatch] = useReducer(reducer, initialState);
dispatch({ type: 'ADD_TO_CART', payload: item });
```

6. Redux (Overview)

- Predictable state container for JS apps.
- Central store holds the entire state.
- Uses: Store, Actions, Reducers.
- Best for large scale applications.



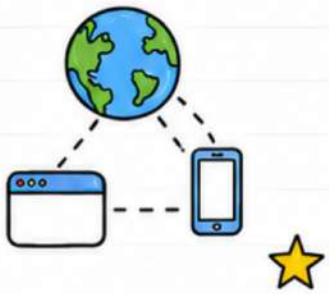
7. When to Use What?

Need	Use
Local component state	useState
Share state between components	Lift State Up
Global state (small to medium)	Context API
Complex state logic	useReducer
Large scale app, many features	Redux

Key Takeaway

- Choose the simplest solution that solves your problem.
- Don't over-engineer state management.
- Good state management = Clean, Scalable & Maintainable React Apps.

08. API CALLS



APIs allow our React app to communicate with servers to fetch or send data and build dynamic applications.

1. Fetch API (Built-in)

- Built-in in browsers.
- Returns a Promise.
- Support async/await.
- Lightweight.

```
async function getData() {
  const res = await fetch('https://api.example.com/data');
  const data = await res.json();
  return data;
}
```

fetch() returns a Response object.

2. Axios (Popular Library)

- Third-party library.
- Cleaner syntax.
- Automatic JSON parsing.
- Better error handling.

```
npm install axios
```

```
import axios from 'axios';

async function getData() {
  const res = await axios.get('https://api.example.com/data');
  return res.data;
}
```

res.data contains the response data.

3. Using useEffect() for API Calls

- useEffect runs after the component renders.
- Perfect place to make API calls.

```
import { useState, useEffect } from 'react';
function Users() {
  const [users, setUsers] = useState([]);

  useEffect(() => {
    const fetchUsers = async () => {
      const res = await fetch('https://jsonplaceholder.typicode.com/users');
      const data = await res.json();
      setUsers(data);
    };
    fetchUsers();
  }, []);   // Empty dependency array => run once
  return (
    <ul>
      {users.map(user => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  );
}
```

Empty [] means run only on mount.

4. Loading State

- Show loader while data is being fetched.
- Improves user experience.

```
const [loading, setLoading] = useState(true);

useEffect(() => {
  const fetchData = async () => {
    const res = await fetch(url);
    const data = await res.json();
    setData(data);
    setLoading(false);
  };
  fetchData();
}, []);
```

5. Error Handling

- Always handle errors to avoid app crashes.
- Show user-friendly messages.

```
const [error, setError] = useState(null);

try {
  const res = await fetch(url);
  if (!res.ok) throw new Error('Something went wrong!');
  const data = await res.json();
  setData(data);
} catch (err) {
  setError(err.message);
}
```

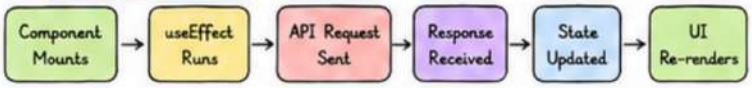
Handle errors gracefully :)

6. Displaying Data

- Once data is fetched, store in state and display using map().
- Use keys while rendering lists.

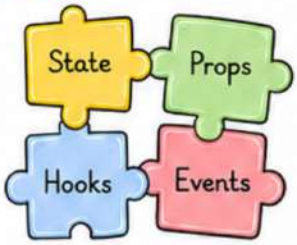
```
return (
  <div>
    {loading && <p>Loading...</p>}
    {error && <p style={{color:'red'}}>{error}</p>}
    {!loading && !error && (
      <ul>
        {data.map(item => (
          <li key={item.id}>{item.title}</li>
        ))}
      </ul>
    )}
  </div>
);
```

7. API Call Flow



Key Takeaway

- APIs make apps dynamic.
- useEffect is your best friend for API calls.
- Always handle loading & errors.
- Keep UI responsive & user-friendly.



09. PRACTICAL EXAMPLES

Let's see some practical examples that combine multiple concepts we've learned.

1. Todo List App (Mini Project)

- Add, delete and mark tasks.
- Uses useState, events and conditional rendering.
- Data is stored in state.

```
function TodoApp() {
  const [todos, setTodos] = useState([]);
  const [input, setInput] = useState("");

  const addTodo = () => {
    if(input.trim() === "") return;
    setTodos([...todos, { id: Date.now(), text:input, done:false}];
    setInput("");
  };
  const toggleTodo = (id) => {
    setTodos(todos.map(todo =>
      todo.id === id ? { ...todo, done: !todo.done } : todo
    ));
  };
  // ... UI code (input, list, buttons)
}
```

This component manages the whole todo list using local state.

2. User Card with API

- Fetch user data from API and display it in cards.
- Shows loading and error states.

```
function UserCard() {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch('https://jsonplaceholder.typicode.com/users/1')
      .then(res => res.json())
      .then(data => { setUser(data); setLoading(false); })
      .catch(err => { console.log(err); setLoading(falee); });
  }, []);

  if (loading) return <p>Loading...</p>;
  if (!user) return <p>Something went wrong!</p>;

  return (
    <div className="card">
      <h3>{user.name}</h3>
      <p@{user.username}</p>
      <p>{user.email}</p>
    </div>
  );
}
```

useEffect runs on mount and fetches the data.

3. Controlled Form

- Handle form inputs using state.
- Submit data and reset the form.

```
function ContactForm() {
  const [form, setForm] = useState({ name: "", email: "", message: "" });

  const handleChange = (e) => {
    setForm({ ...form, [e.target.name]: e.target.value });
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    alert(`Thanks ${form.name}! We received your message.`);
    setForm({ name: "", email: "", message: "" });
  };

  return (
    <form onSubmit={handleSubmit}>
      <input name="name" value={form.name} onChange={handleChange} placeholder="Name" />
      <input name="email" value={form.email} onChange={handleChange} placeholder="Email" />
      <textarea name="message" value={form.message} onChange={handleChange} placeholder="Message" />
      <button type="submit">Send</button>
    </form>
  );
}
```

Each input is controlled by React state.

4. Dark / Light Theme Toggle

- Toggle between dark and light mode.
- State + conditional classNames.

```
function ThemeToggle() {
  const [dark, setDark] = useState(false);

  return (
    <div className={dark ? 'theme dark' : 'theme light'}>
      <button onClick={() => setDark(!dark)}>
        {dark ? "☀️ Light Mode" : "🌙 Dark Mode"}
      </button>
      <p>Hello, {dark ? 'Night Owl 🦉' : 'Early Bird 🐦'}!</p>
    </div>
  );
}
```

CSS Idea:

```
.theme.light { background: #fffff; color: #111; }
.theme.dark { background: #111;    color: #eee; }
```

5. Error Handling

- Always handle errors to avoid app crashes.
- Show user-friendly messages.

```
const [error, setError] = useState(null);

try {
  const res = await fetch(url);
  if (!res.ok) throw new Error("Something went wrong!");
  const data = await res.json();
  setData(data);
} catch (err) {
  setError(err.message);
}
```

Handle errors gracefully :)

6. Displaying Data

- Once data is fetched, store in state and display using map().
- Use keys while rendering lists.

```
return (
  <div>
    {loading && <p>Loading...</p>}
    {error && <p style={{color:'red'}}>{error}</p>}
    {!loading && !error && (
      <ul>
        {data.map(item => (
        <li key={item.id}>{item.title}</li>
        ))}
      </ul>
    )}
  </div>
);
```

- Item 1
- Item 2
- Item 3

7. API Call Flow

Component Mounts → useEffect Runs → API Request Sent → Response Received → State Updated → UI Re-renders

Key Takeaway

- APIs make apps dynamic.
- useEffect is your best friend for API calls.
- Always handle loading & errors.
- Keep UI responsive & user-friendly.

Practice daily → Build projects → Get better → Enjoy the journey!

@thesanskaarsingh

10/10

10. SUMMARY & BEST PRACTICES

Practice
Build
Improve
Repeat!

You've learned a lot! Here's a quick recap and some best practices to help you build better React applications.

1. Key Concepts Recap

- Components are the building blocks.
- Props pass data from parent to child.
- State manages data that changes.
- Hooks let us use state and other React features.
- Events handle user interactions.
- Conditional rendering shows content based on conditions.
- Lists and keys help render collections efficiently.
- React Router helps build multi-page apps.
- APIs bring dynamic data to our applications.

2. Best Practices

- Keep components small and focused.
- Use meaningful names for variables and functions.
- Lift state up only when necessary.
- Avoid prop drilling, use Context API instead.
- Handle loading and error states in API calls.
- Keep your code clean, readable and reusable.
- Use keys in lists to help React identify items.
- Test your components and handle edge cases.

Clean Code
Better UX
Happy Users

3. Code Snippets Quick Reference

useState

```
const [count, setCount] = useState(0);
setCount(count + 1);
```

useEffect

```
useEffect(() => {
  // side effect
}, []);
```

Conditional Rendering

```
{isLoggedIn ? (
  <p>Welcome!</p>
) : (
  <p>Please Login</p>
)}
```

List Rendering

```
{items.map(item => (
  <li key={item.id}>{item.name}</li>
))}
```

API Call with fetch

```
useEffect(() => {
  fetch('https://api.example.com/data')
    .then(res => res.json())
    .then(data => setData(data));
}, []);
```

Fetch Data

4. Project Ideas to Practice

- To-Do List App
- Weather App (using OpenWeather API)
- User Profile Card
- Notes App with Local Storage
- E-commerce Product List

5. Learning Roadmap (What's Next?)

Keep Learning	Learn advanced concepts like Context, Reducer, Custom Hooks.
Build More	Build real projects and improve your skills.
Explore Further	Learn TypeScript, Next.js, Testing (Jest, RTL)
Go Production	Deploy apps, optimize performance and keep growing!

6. Error Handling

- Always handle errors to avoid app crashes.
- Show user-friendly messages.

```
const [error, setError] = useState(null);
try {
  const res = await fetch(url);
  if (!res.ok) throw new Error('Something went wrong!');
  const data = await res.json();
  setData(data);
} catch (err) {
  setError(err.message);
}
```

Handle errors gracefully :)

7. API Call Flow

Component Mounts → useEffect Runs → API Request Sent → Response Received → State Updated → UI Re-renders

You've made it to the end!
Keep coding, keep building,
and most importantly,
enjoy the process.

Key Takeaway

- React is all about components, state and props.
- Practice regularly and build projects.
- Use best practices to write clean and scalable code.
- Never stop learning and experimenting!

Code Today → Debug Tomorrow → Ship Friday → Repeat Forever!